



FALL 2026

SEG3102 Software Design and Architecture

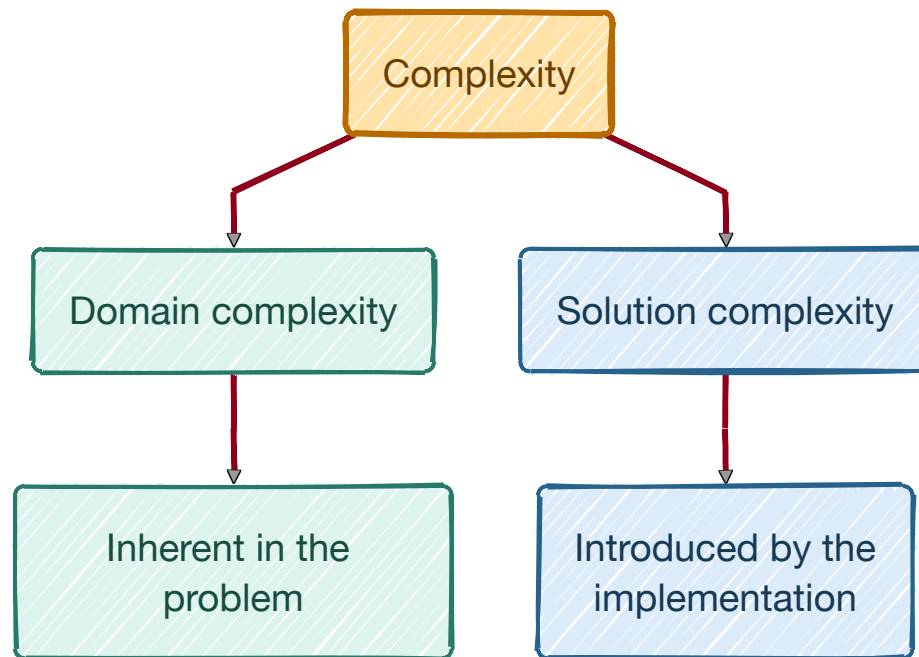
Introduction to Domain-Driven Design

Slide content used in this course is adapted from Prof. Stéphane Somé.

Last Lecture

- Software design involves **proposing, representing, and refining** solutions, from requirements and constraints to a realizable plan
- Design decisions range from **architecture** to **detailed design**
- Architectural design models the system from **multiple perspectives**: module, concurrency, deployment
- Design happens **up front, as the code grows, or both**; postponed decisions accumulate **technical debt**
- Good design captures complexity inherent in the **problem domain** without introducing unnecessary solution complexity

Where is all this complexity coming from?



Does the problem give us the complexity, or did we create it?

Some complexity was already there

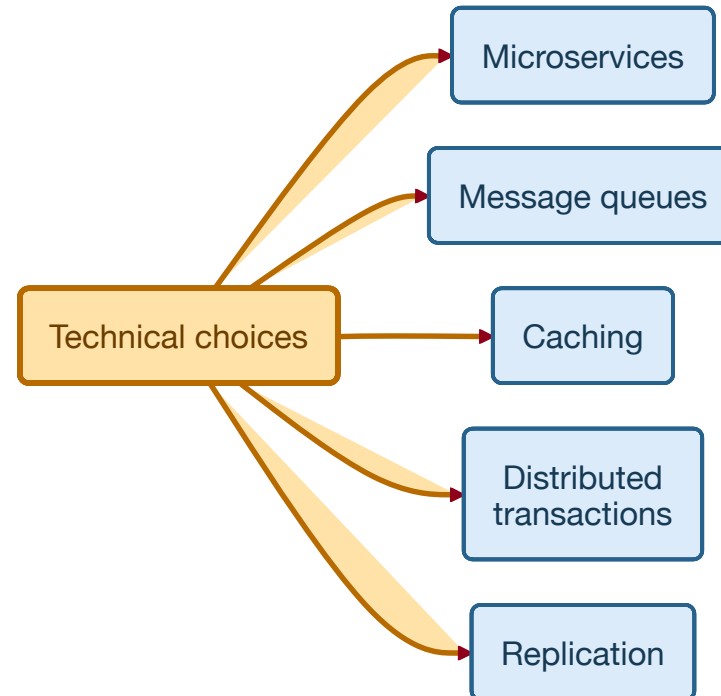
Domain complexity exists because the problem itself is complex

- Different sellers have different return rules
- Inventory is distributed across warehouses
- One order may produce several shipments
- Some products cannot ship to some regions
- Refund rules depend on payment method, seller, product type, and order state

Complexities are independent of how we implement

Some complexity came with our solution

Solution complexity is introduced by how we build the software



Good design represents **necessary domain complexity** while avoiding **unnecessary solution complexity**.

Exercise: Domain or solution complexity?

Classify each source of complexity in **course registration**.

- Prerequisites must be satisfied.
- Some seats are reserved for particular programs.
- Courses cannot overlap in a student's schedule.
- Use a particular service caches course availability.
- Create a message queue to communicates registration events.
- Registration data is replicated across servers.

Exercise: Domain or solution complexity?

Go to wooclap.com and use the code **YBKTOZLZ**

Classify each source of complexity in course registration: domain / solution

The correct answers are

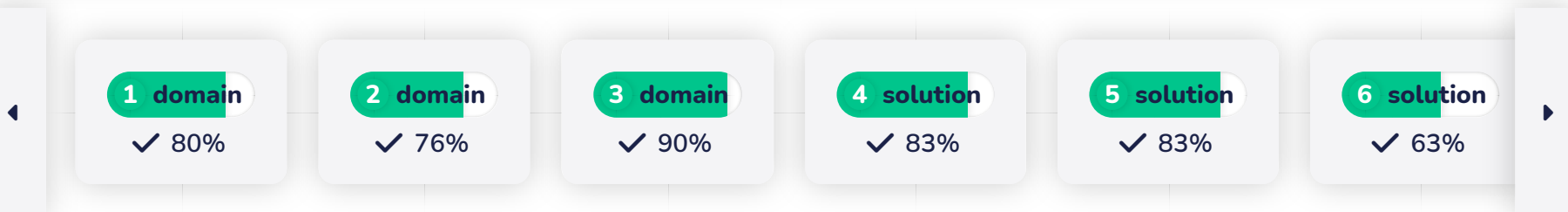
Prerequisites must be satisfied: **1 domain**

Some seats are reserved for particular programs: **2 domain**

Courses cannot overlap in a student's schedule: **3 domain**

Use a particular service caches course availability: **4 solution**

Create a message queue to communicate registration events: **5 solution**



Exercise: Follow the source

Domain

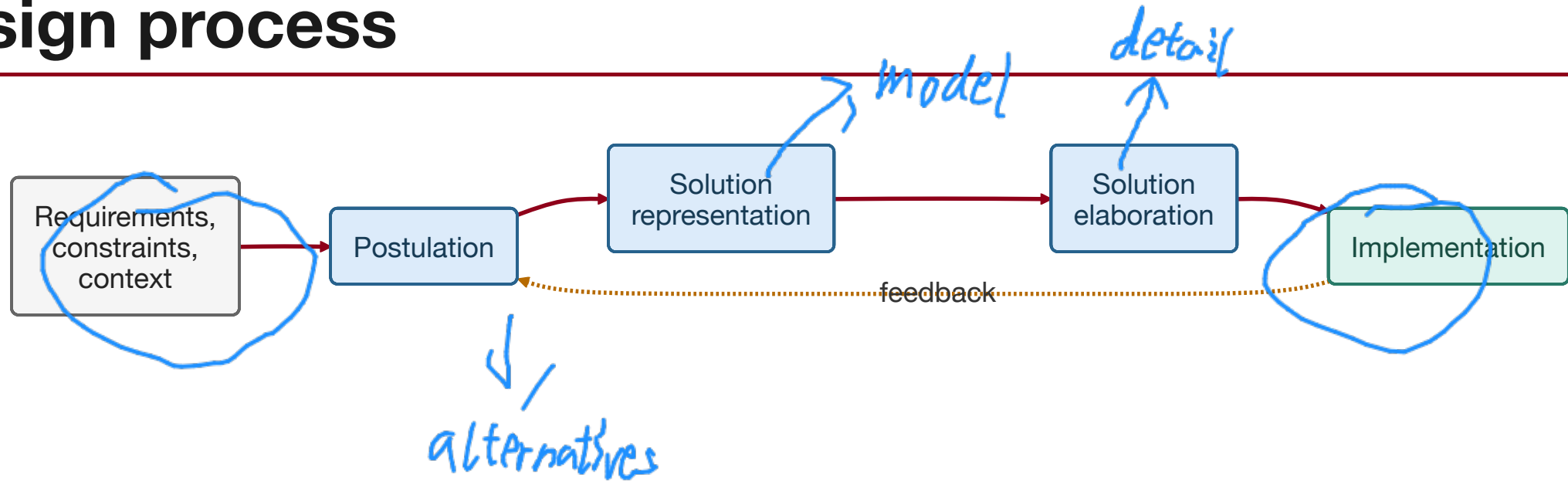
- Prerequisites
- Reserved seats
- Schedule conflicts

Solution

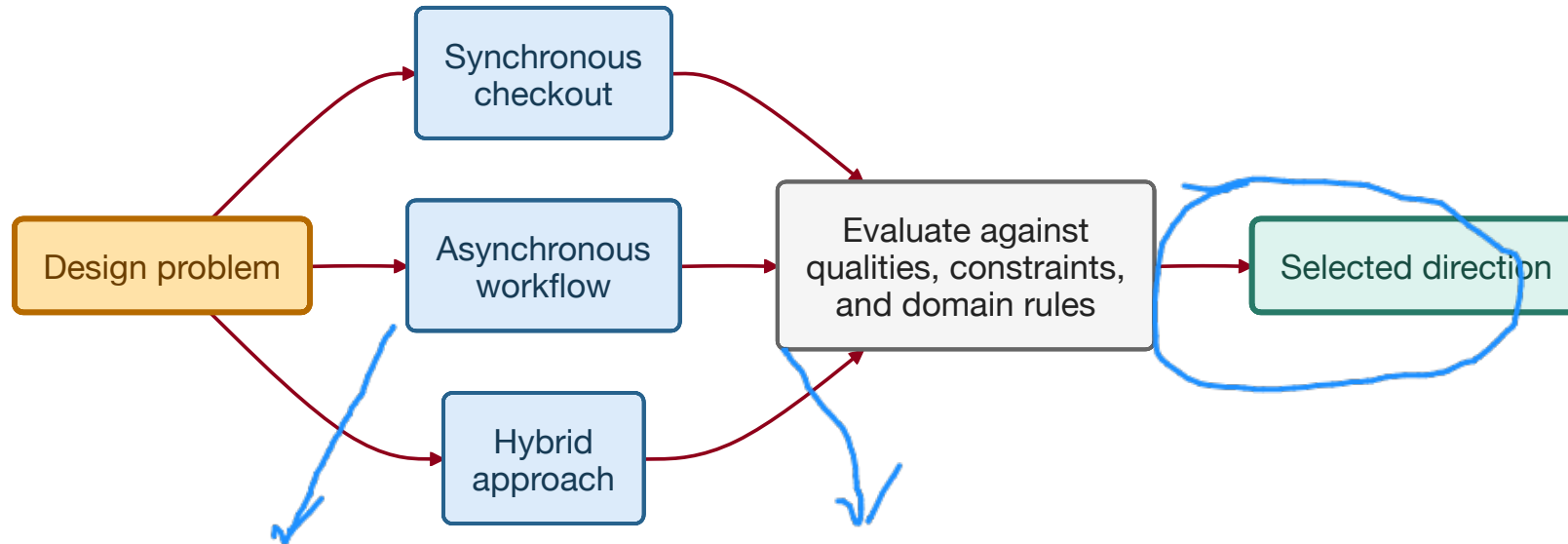
- Caching
- Message queues
- Replication

Domain Driven design
↳ verb → design process

Design process



Postulation



Diversification: generate genuinely different candidate solutions

Convergence: compare the candidates and commit to a defensible direction

Solution representation and elaboration

Solution representation

Make the selected direction visible and discussable through different types of **models**

- architecture views;
- interface contracts;
- prototypes;
- recorded decisions.

Solution elaboration

Refine the models into actual **system**

- component responsibilities;
- detailed designs;
- algorithms and data structures;
- working code.

Elaboration reveals new issues, so the process may return to postulation

Outline

1. What is a domain?
2. Domain models
3. Use cases
4. Ubiquitous language
5. Domain-Driven Design (DDD)
6. Behavior-driven development and Gherkin

How much domain understanding do we need to start design?

**Customer · Product · Seller · Order · Payment · Refund Order
Item · Inventory · Warehouse · Shipment · Return**

- What exactly does each concept mean?
- Which rules govern it?
- Which concepts belong together?
- Does every team mean the same thing by “Product” or “Order”?

Before we can design a solution, we need a usable model of the domain.

How do we connect the domain to the design process?

With Domain-Driven Design (DDD)

How do we understand the domain well enough to represent it?

What is a domain?

Meaning of one word



My order is the three things I bought yesterday. I want to know when it arrives.

↓
product

Order? I see two pick lists, one for each warehouse.



An order is an invoice with tax and a payment record

↗ invoice

↘ inventory

The word “Order” has three meanings.

```
1 val order = Order(orderId)
```

So, what does the variable `order` in the code mean?

What is a domain?

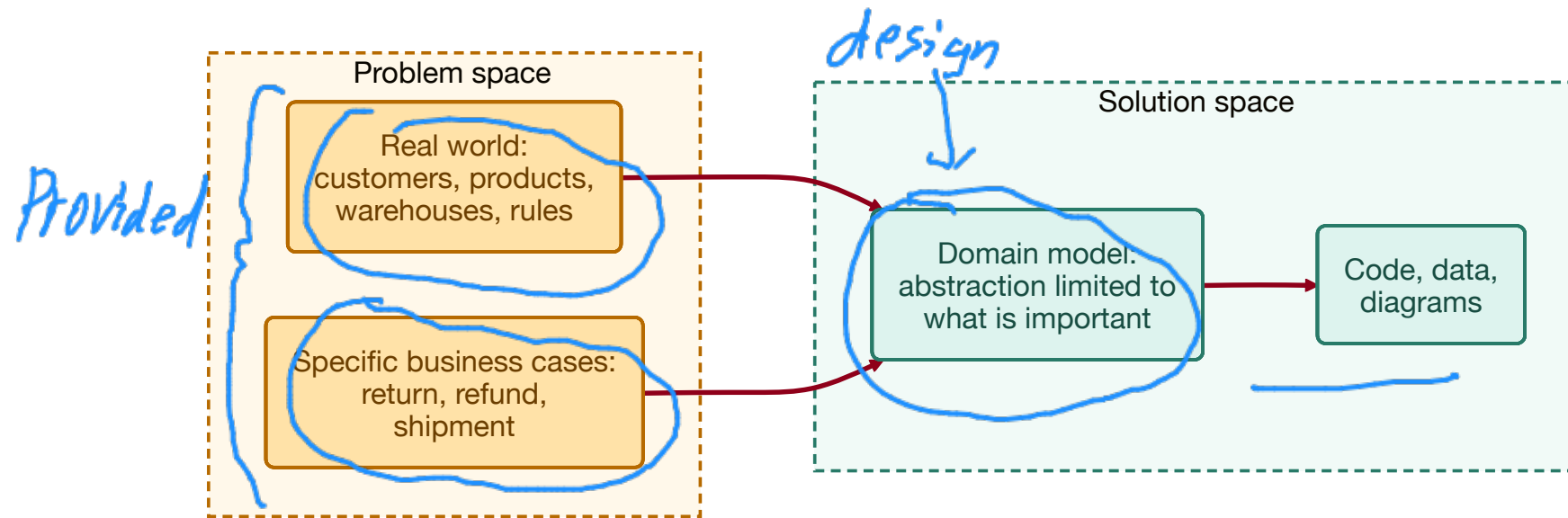
Domain

The **domain** is the subject area of an application: the problem space for which a solution is created.

- Can be tangible (shipping parcels) or intangible (pricing insurance)
- Exists before the software and will outlive it
- Must be reflected in the code, or the code is not solving the right problem
Validation

Our example domain: selling goods online and getting them to the customer

Problem space and solution space

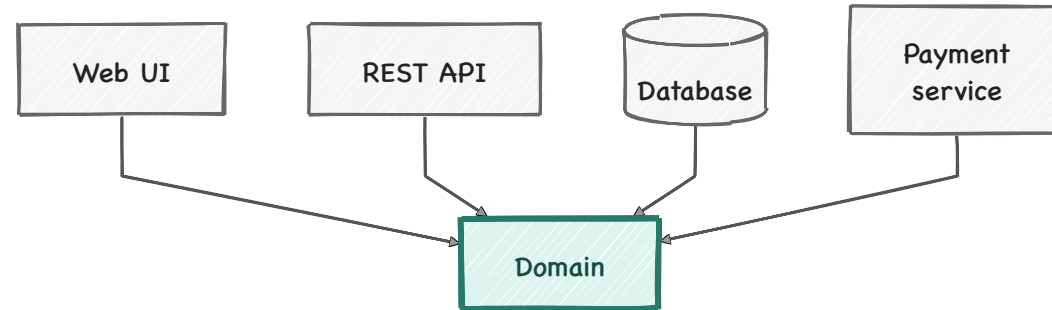


Domain analysis is the collaborative process by which developers and domain experts capture the realities of the problem space in a domain model in the solution space.



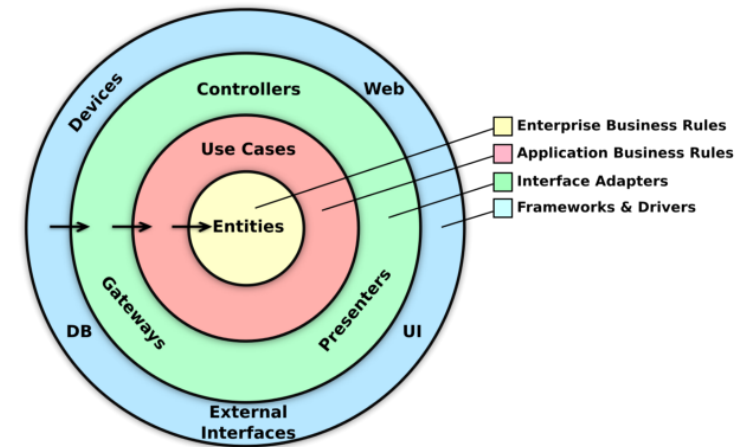
E.g., EventStorming

The domain is the most stable part of the system



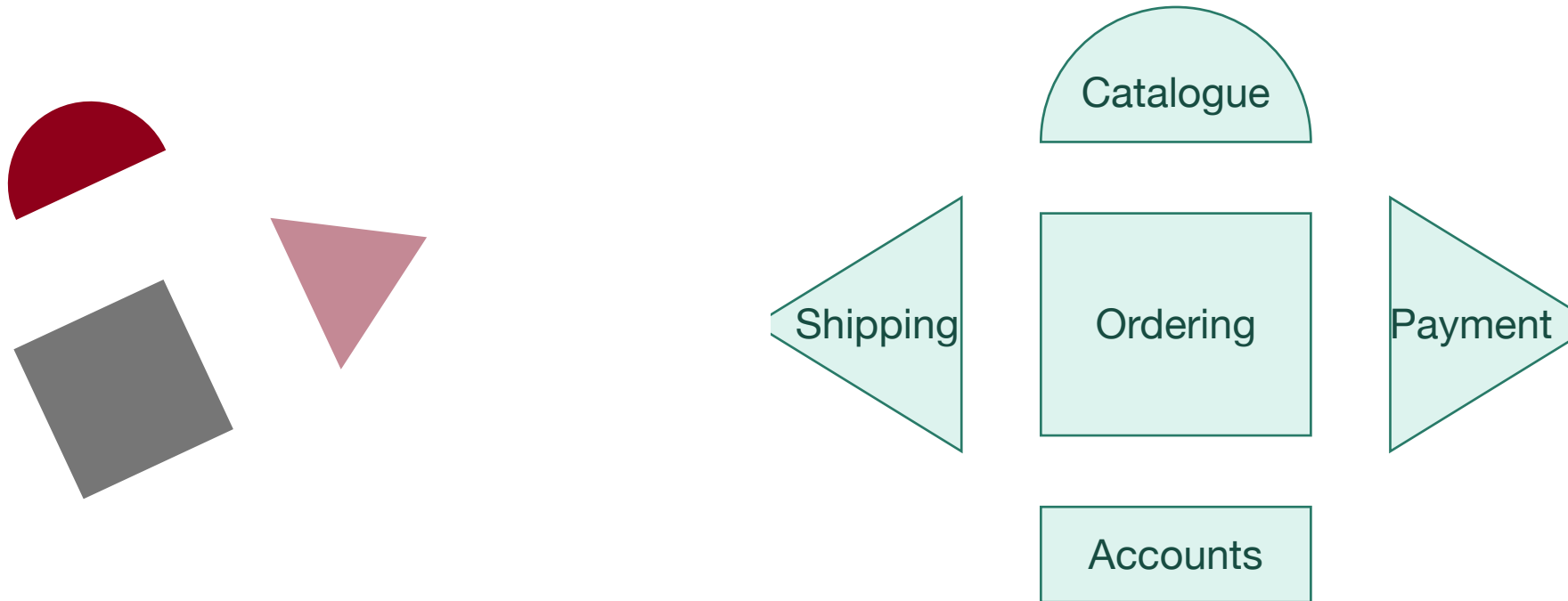
- Front-end frameworks change every few years
- Databases and interfaces get replaced
- AI agents *may* write all the code

The rules for selling goods change slowly



Subdomains

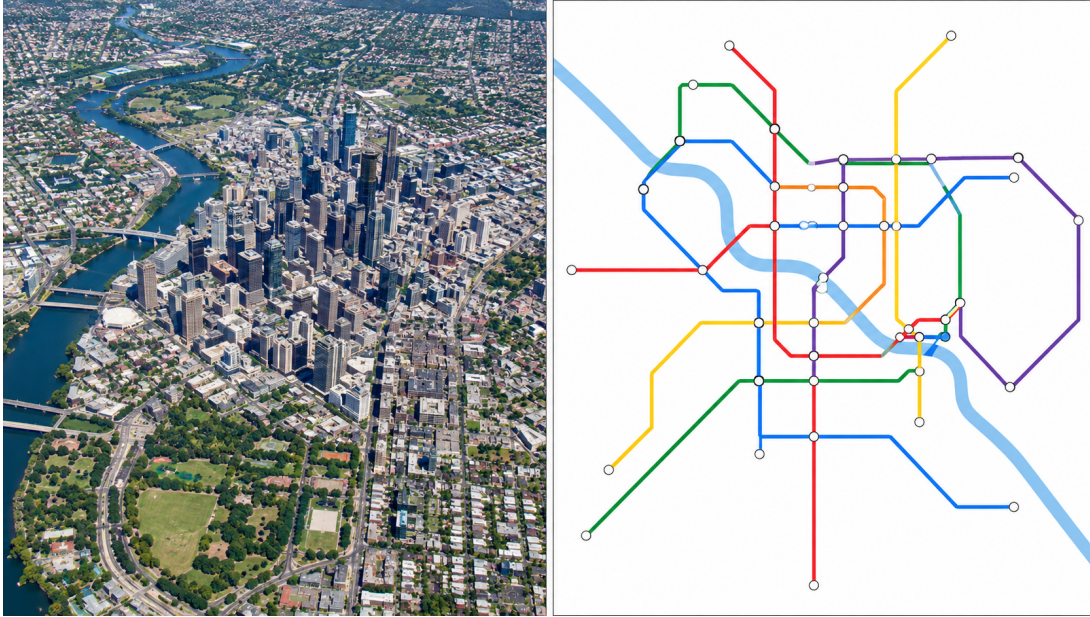
A whole domain is difficult to grasp at once. It is made up of areas of capability, or **subdomains**. We can **decompose** a domain into multiple subdomains.



Not all of these matter equally to the project, ranking them is part of **strategic design**

Domain models

Abstraction is everywhere



The real world of our shop contains:

- products, prices, and suppliers
- warehouse staff and their shifts
- Warehouse dimensions, shelves
- the weather on delivery day

Which parts of information is necessary for the software?

Keep what answers the question, drop the rest.

Domain model

Domain model

A **domain model** is an abstraction of the domain that includes the concepts, relationships, and rules relevant to solving the problem, and leaves out the rest.

- Built from specific **business cases**
- Limited to what is important for the **software**
- Shared by **developers and domain experts**

UML Class Diagram

A model is not one artifact. It is expressed in diagrams, in code, in data schemas, and in the way the team talks

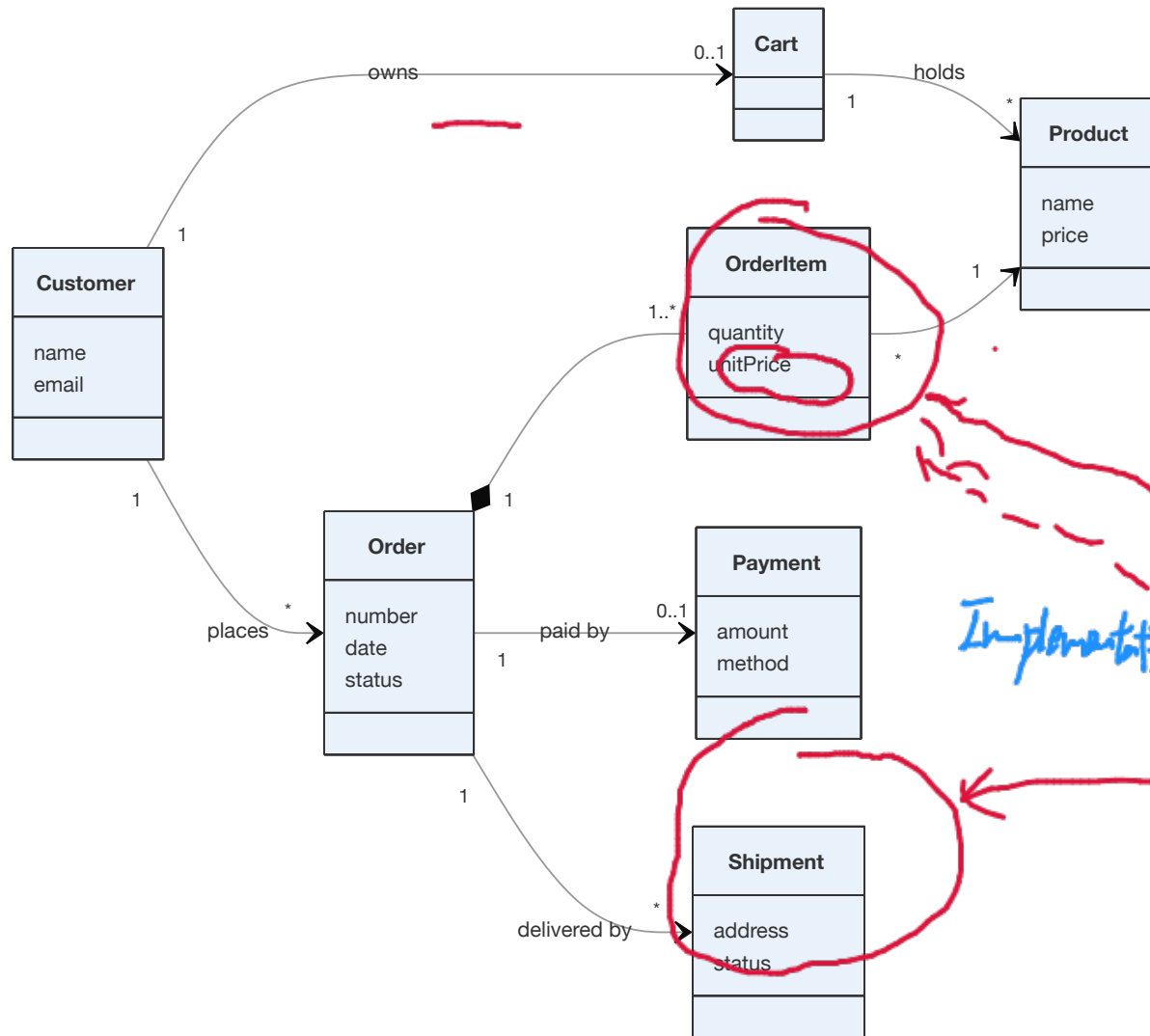
The domain: an online shop



We run an online shop. Each product has a name and a price, which can change over time. Customers browse the catalogue and add products to their cart. At checkout, the cart becomes an order, with a separate line for each product showing the quantity and the price charged at the time of purchase. Customers pay for the full order using a card or gift card. We ship from two warehouses, so larger orders may arrive in several parcels, each with its own tracking status.

Which concepts, relationships, and rules does this description contain?

Domain model for the online shopping system



Concepts: customer, product, cart, order, payment, shipment.

Relationships: a customer places orders, an order is made of items, each item refers to one product.

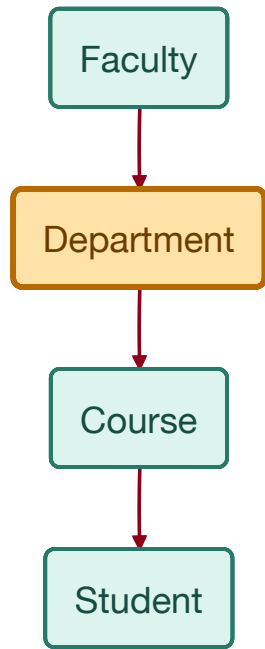
Rules: (sometimes represented in constraint languages)

- an order has at least one item
- an item keeps the unit price at the time of purchase
- one order may be delivered in several shipments

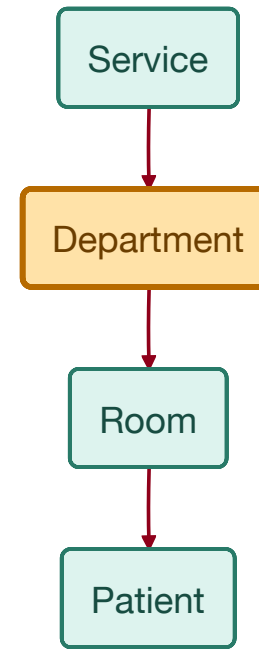
Implementation

Model gives meaning to concepts

University



Hospital



Both models have a **Department**. But it means different things: a university department offers courses, a hospital department has rooms and patients.

Exercise: What is in the model?

Here is what a domain expert told us about the shopping and shipping processes. Which concepts belong in the **domain model** of the online shopping system, and which do not?

Every product has a name and a price, and comes from a supplier who ships it to our warehouse in Ottawa. John Doe works in the warehouse, starts at 7 am and drives a forklift. Customers add products to a cart. When they check out, an order is created and the payment is taken through the company's payment service, which runs on Java servers. Delivery takes two to five business days.

List the concepts you would keep

Exercise: What is in the model?

Go to wooclap.com and use the code **YBKTOLZ** 

Here is what a domain expert told us about the shopping and shipping processes. Which concepts belong in the **domain...



No answers yet. They'll show up here as they're submitted.

Exercise: Answers

In the model

- Product, name, price
- Supplier
- Warehouse
- Customer, Cart
- Order, Payment
- Delivery and its lead time

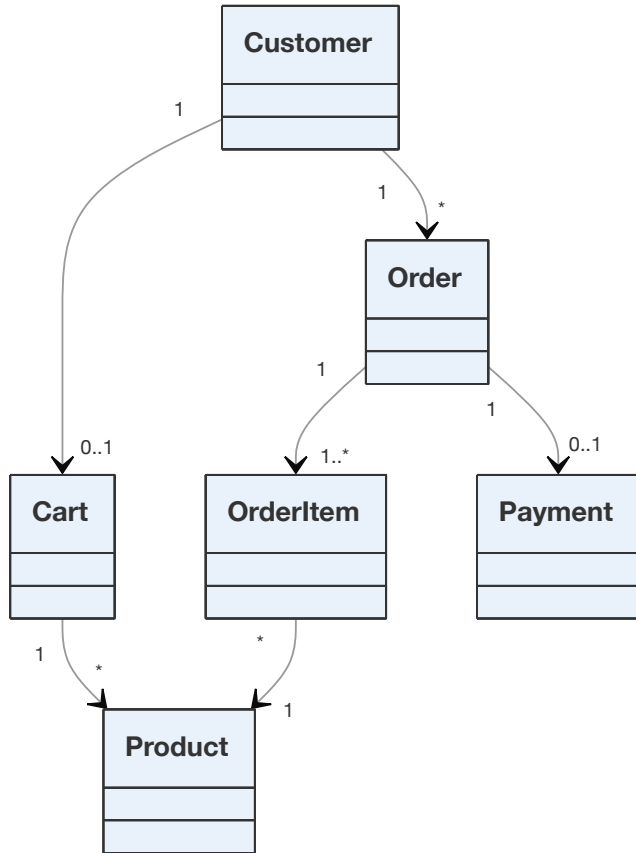
Not in the model

- John Doe and the start time
- The forklift (Maybe relevant if IoT is involved)
- Servers (solution, not domain)
- Ottawa? Only if location matters to a rule

“Delivery takes two to five business days” is a **domain rule**. Rules belong in the model too.

Use cases

Conceptual models do not capture behavior



The structural model tells us what exists and how things relate

It does not tell us:

- Can a customer cancel an order after it has shipped?
- What happens if payment fails halfway?
- Who is allowed to change a price?

domain expert

To capture behavior, we describe how actors can reach their goals.

Use cases

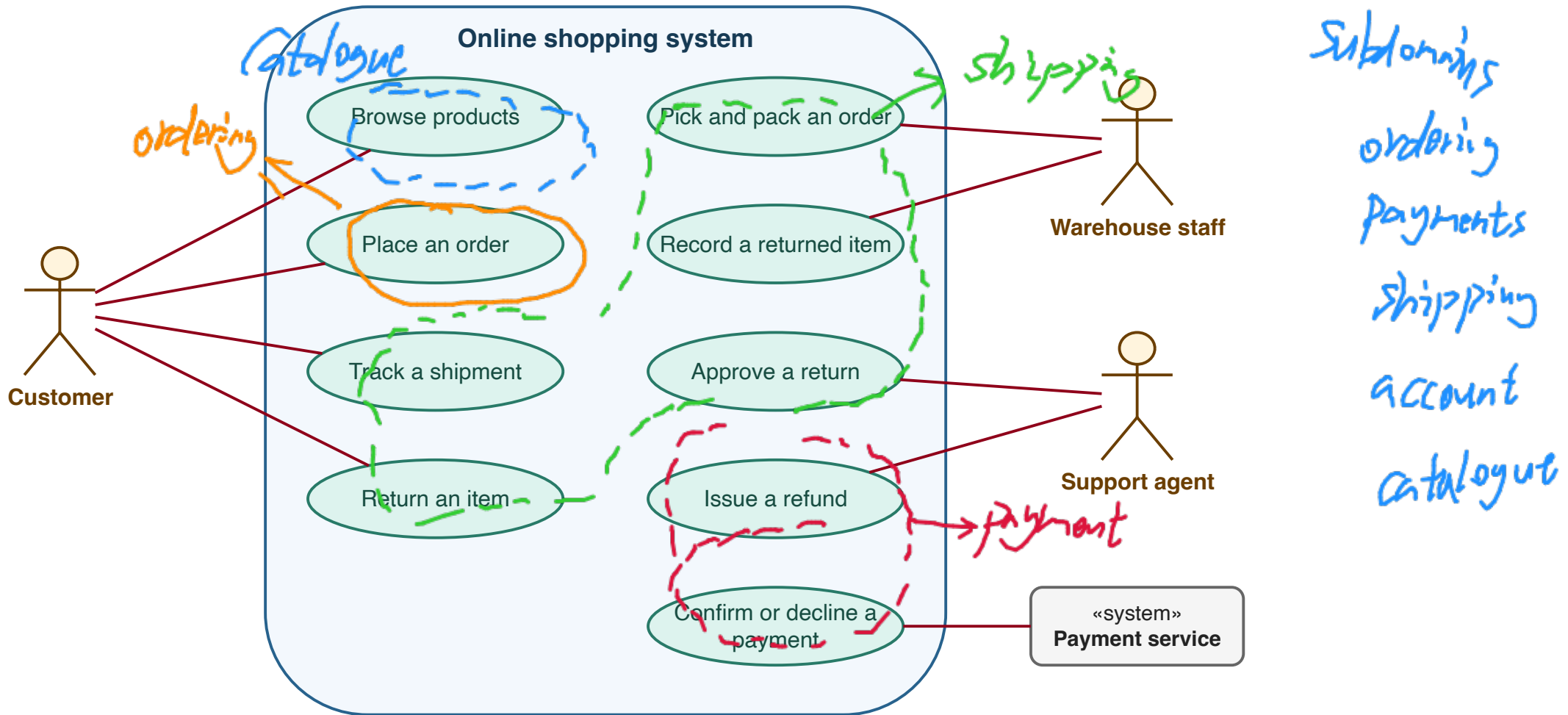
Use case

A **use case** is a collection of related success and failure scenarios that describe an actor using the system to achieve a goal.

Actor	Goals
Customer	Browse products, place an order, return an item, track a shipment
Warehouse staff	Pick and pack an order, record a returned item
Support agent	Approve a return, issue a refund
Payment service	Confirm or decline a payment

An actor can be a person or another system. What matters is that it has a goal.

The use case diagram shows the boundary



Use case: place an order

```
1 Use case:      Place an order
2 Primary actor:  Customer
3 Preconditions:  The customer is signed in and the cart is not empty.
4
5 Main success scenario:
6   1. The customer asks to check out the cart.
7   2. The system shows an order summary: items, prices, delivery address.
8   3. The customer confirms the address and chooses a payment method.
9   4. The system reserves the items in inventory.
10  5. The system requests the payment from the payment service.
11  6. The payment service confirms the payment.
12  7. The system places the order and empties the cart.
13  8. The system sends an order confirmation to the customer.
14
15 Extensions:
16  4a. An item is no longer in stock:
17     4a1. The system removes it and shows the updated summary.
18     4a2. The customer continues with the rest or cancels.
19  6a. The payment is declined:
20     6a1. The system releases the reservation and informs the customer.
21     6a2. The customer chooses another payment method or cancels.
22
```

Domain rules are hidden in the extensions

```
1  4a. An item is no longer in stock:
2      4a1. The system removes it and shows the updated summary.
3      4a2. The customer continues with the rest or cancels.
4  6a. The payment is declined:
5      6a1. The system releases the reservation and informs the customer.
6      6a2. The customer chooses another payment method or cancels.
7  3a. The address is in a region the item cannot ship to:
8      3a1. The system flags the item and asks for another address.
9      3a2. The customer changes the address or removes the item.
```

Each extension can be **business rule**:

- stock is reserved before payment, and released if payment fails;
- an order is never placed for an item that cannot ship to its destination.

Use cases name the domain concepts

The **customer** asks to check out the **cart**.
The system shows an **order summary** with **items**, **prices**, and a **delivery address**.
The customer chooses a **payment method**. The system **reserves** the items in **inventory**, requests the **payment**, places the **order**, and sends an **order confirmation**.

Candidate concepts:

- Customer, Cart
- Order, Order item, Price
- Delivery address
- Payment method, Payment
- Inventory, Reservation
- Order confirmation

Use case description may reveal new domain concepts not in the initial requirement

Exercise: Use case extensions?

The domain expert describes **returns** as a use case description:

A customer can return an item within 30 days of delivery. When the warehouse receives it and confirms it is unused, we refund the original payment method. Some items, such as opened software, cannot be returned.

Here is the main success scenario of **Return an item**:

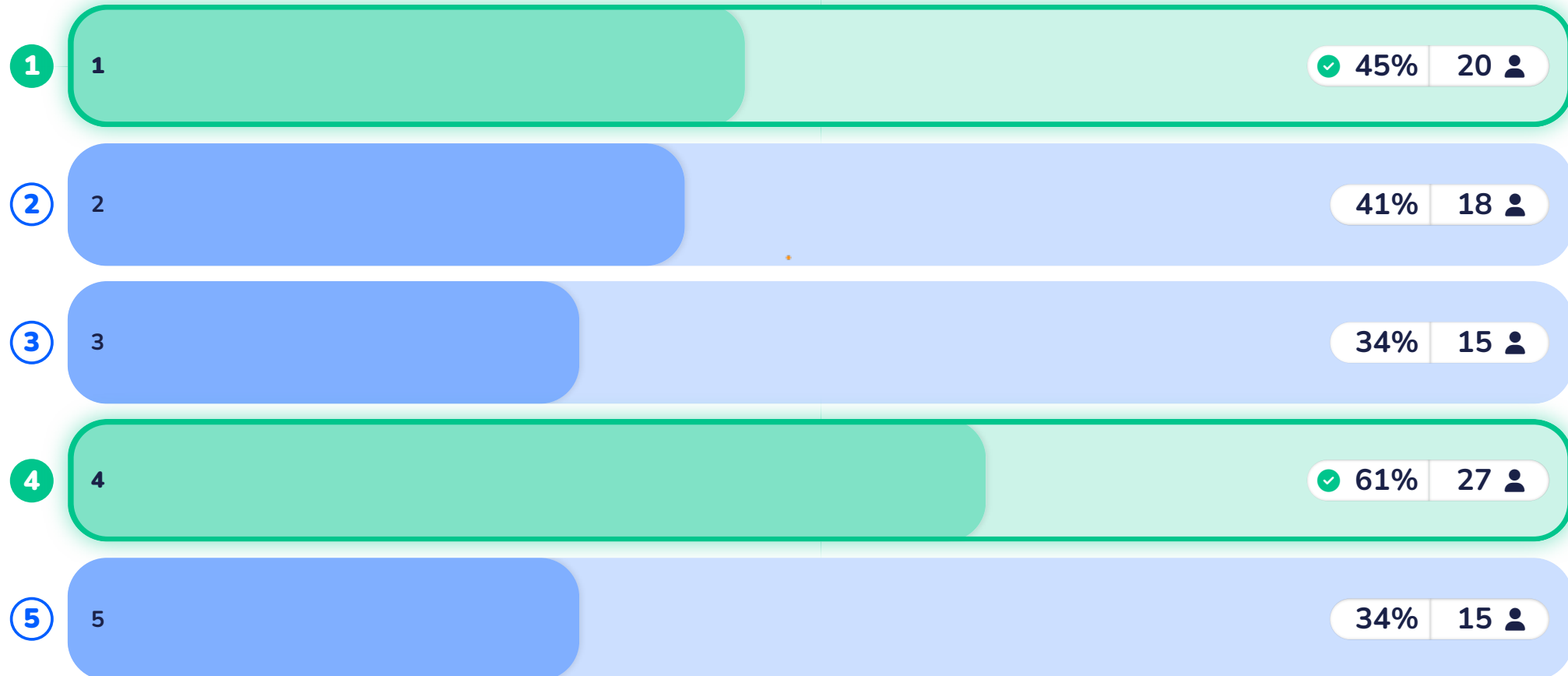
- 1 1. The customer selects a delivered item and gives a reason.
- 2 2. The system creates a return request and issues a shipping label.
- 3 3. The customer ships the item back.
- 4 4. The warehouse receives the item and confirms it is unused.
- 5 5. The system refunds the original payment method.

Find two extensions in the expert's description. At which step does each one branch off?

Exercise: Use case extensions?

Go to wooclap.com and use the code **YBKTOZLZ**

> A customer can return an item within 30 days of delivery. When the warehouse receives it and confirms...



Exercise: Answers

```
1 Use case:      Return an item
2 Primary actor: Customer
3 Preconditions:  The customer has logged in and has a delivered item.
4 Main success scenario:
5     1. The customer selects a delivered item and gives a reason.
6     2. The system creates a return request and issues a shipping label.
7     3. The customer ships the item back.
8     4. The warehouse receives the item and confirms it is unused.
9     5. The system refunds the original payment method.
10
11 Extensions:
12     1a. More than 30 days have passed since delivery:
13         1a1. The system does not offer the return option.
14     1b. The item is not returnable (e.g. opened software):
15         1b1. The system rejects the request and explains why.
16     4a. The warehouse finds the item was used:
17         4a1. The system rejects the refund and notifies the customer.
18
19 Postconditions: The return request is either completed or rejected, and the customer is notified.
```

Ubiquitous language

Translation loses information



When a customer sends an item back, we refund them once the warehouse has checked it.

Got it: a return request requires an authorization, then a memo after inspection.



So I need a `ReturnDTO`, a `RefundProcessorImpl`, and an `inspected` flag on the item row.

These are three sets of vocabularies for one process. Each translation drops a little meaning, and six months later nobody remembers what “return” meant exactly.

Ubiquitous language

Ubiquitous language

The **ubiquitous language** is the common vocabulary of the domain, shared by domain experts and developers, and used in conversations, documents, models, and code.

- **Common terminology** and structure for the whole team
- Binds the **model and the implementation**
- Supports **change**: when the business changes a term, the code follows

The domain model **is** the ubiquitous language. Diagrams, use cases, and classes in code are different views of the same vocabulary.

Binding model and implementation

The use case says

```
1 4. The system reserves the  
2   items in inventory.
```

After aligning the language

```
1 val reservation =  
2   inventory.reserve(order.items)
```

The code says

```
1 stockManager.decrementQty(sku, n)  
2 flags[sku] = "HOLD"
```

- “reserve” is what the domain says
- **Reservation** is now a common concept
- “HOLD” flag and “decrement” are solution terms

If the domain expert cannot read a method name aloud, the name is probably wrong.

Signs the language is missing

- **Two words for one thing:** “client” in the UI, “customer” in the database, “user” in the code
- **One word for two things:** “order” means both a cart at checkout and a shipment
- **Generic names:** *Manager*, *Helper*, *Processor*, *Data*
- Requirements and code written in heavy **technical jargon**
- Developers constantly **translating** meanings to the domain experts

The language is built through **collaboration** between domain experts and developers

Exercise: What are good candidates for the ubiquitous language vocabularies?

Which terms below are good candidate for the **ubiquitous language** of the online shopping system?

- Customer
- Item
- Transaction
- Order confirmation
- Cart
- Status

Exercise: What is in the model?

Go to wooclap.com and use the code **YBKTOLZ** 

Which terms below are good candidates for the ****ubiquitous language**** of the online shopping system?

- ① Customer
- ② Item
- ③ Transaction
- ④ Order confirmation
- ⑤ Cart
- ⑥ Status

Exercise: Answers

Potentially good candidates

- Customer
- Order confirmation
- Cart

Potentially bad candidates

- Item (too generic, is it “order item” or “product”?)
- Transaction (potentially mixed with technical meaning: “payment” or “database transaction”?)
- Status (too generic if used along. Is it “order status” or “shipment status”?)
 - Needs to be used within a context to be meaningful

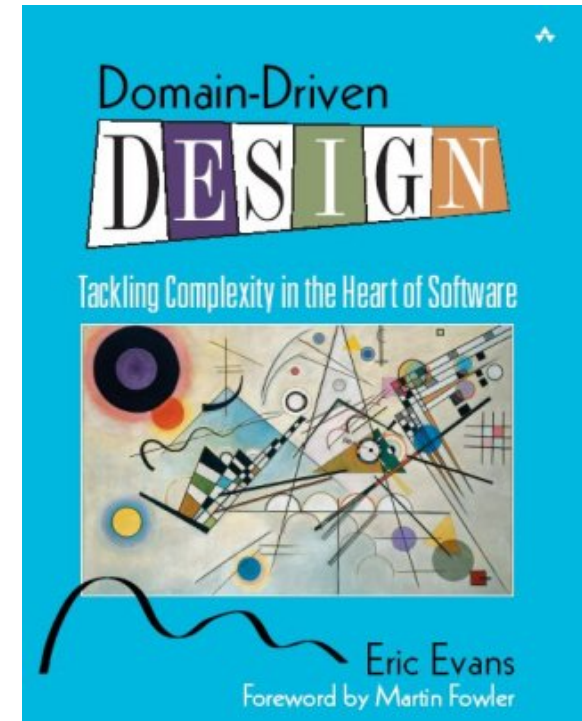
Domain-Driven Design

Domain-Driven Design in one sentence

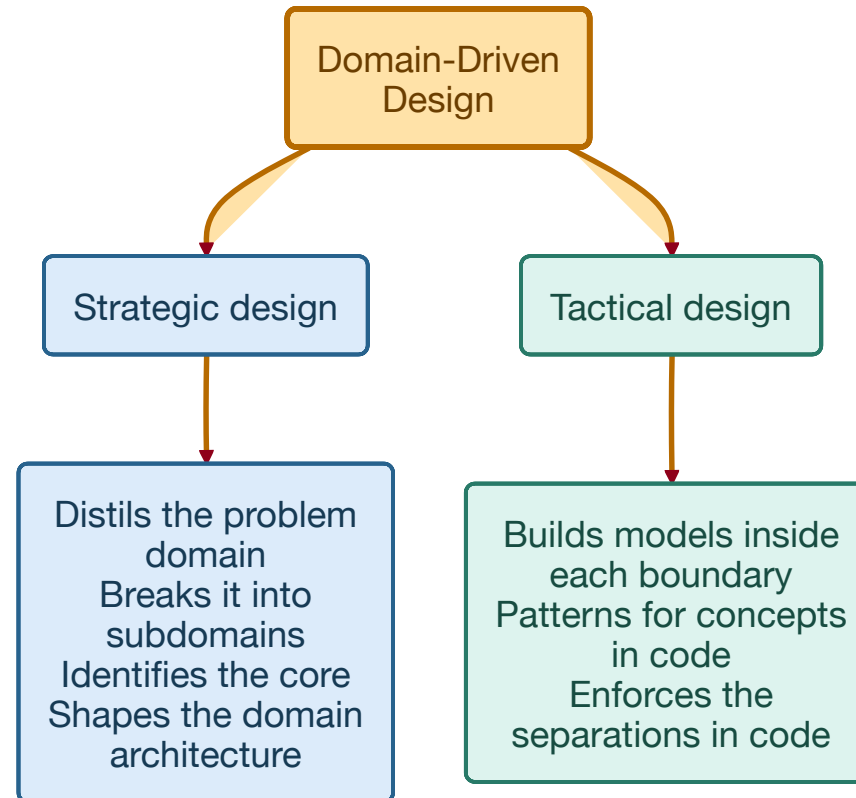
Domain-Driven Design (DDD)

Domain-Driven Design is an approach to software design that places the domain model at the centre: the model reveals what is important, and the code is kept aligned with it.

- Collaboration between domain experts and developers
- Developers map the agreed model directly to code
- A set of principles and patterns, published by Eric Evans in 2003

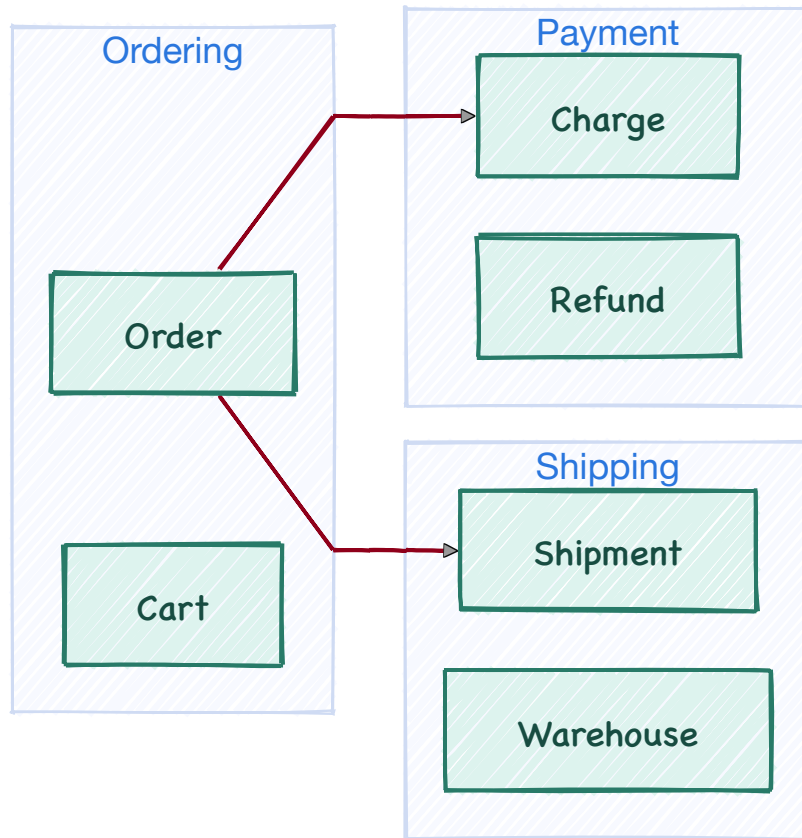


Two levels of DDD: strategic and tactical



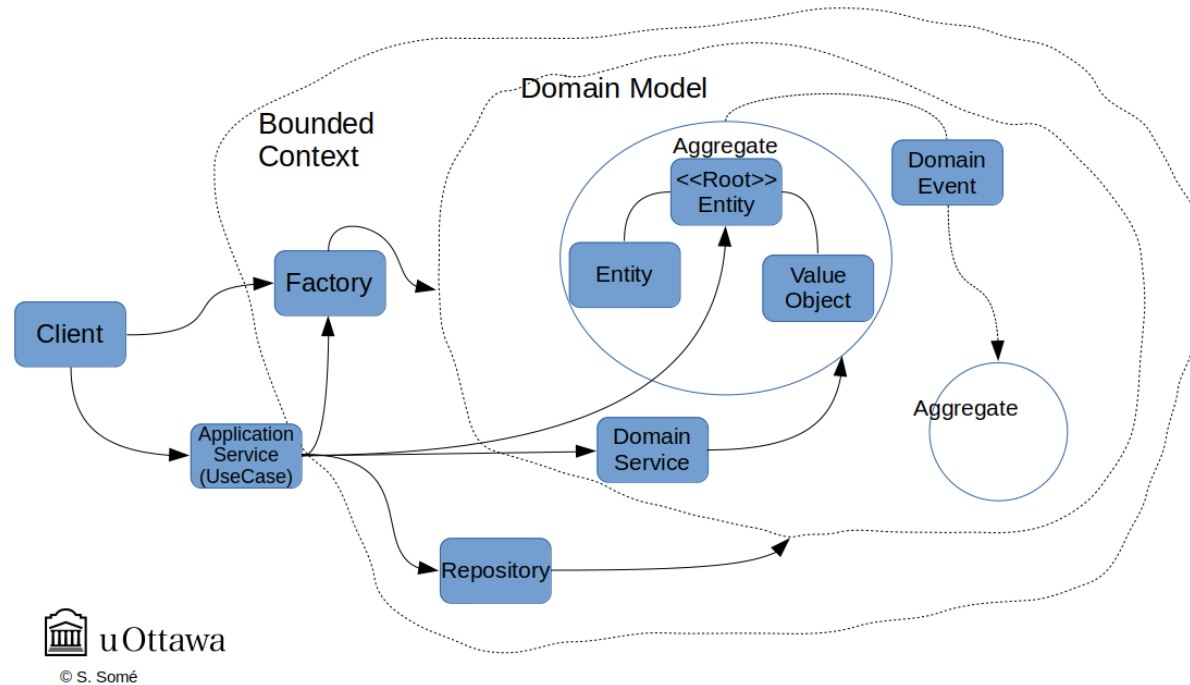
Strategic design **decomposes the domain into boundaries**. Tactical design decides **what the code inside a boundary looks like**.

Strategic design discovers the boundaries



- partition the domain into subdomains;
- rank the partitions by importance to the business;
- draw the boundaries for a single model;
- decide how the boundaries integrate.

Tactical design shapes the code inside a boundary



All the tactical patterns inside one bounded context

For each use case, there is an **application service**:

```

1 // application service
2 class PlaceOrder(/* ... */) {
3     fun execute(id: CustomerId): Order {
4         val cart = carts.of(id)
5         val reservation =
6             inventory.reserve(cart.items)
7         val payment =
8             payments.charge(cart.total)
9         val order = Order.place(
10             cart, reservation, payment)
11         orders.save(order)
12         return order
13     }
14 }
  
```

DDD is not strict

- **Not a framework.** It is a way of thinking about models and boundaries for the problem.
- **Not needed everywhere.** DDD only pays off if the main complexity comes from the domain, i.e., the problem space.
- **Not something developers do alone.** Without domain experts in the room, the “domain model” is just a class diagram.
- **Barely strictly followed.** Few teams apply DDD strictly, but any team building complex software ends up doing some of what it describes.

Behavior-driven development and Gherkin

A tool to build the ubiquitous languages



The use case looks right. But does the system actually release the order reservation when payment fails?

I think so. Let me find the test... it is called `testOrderFlow3`. It checks that no exception is thrown.

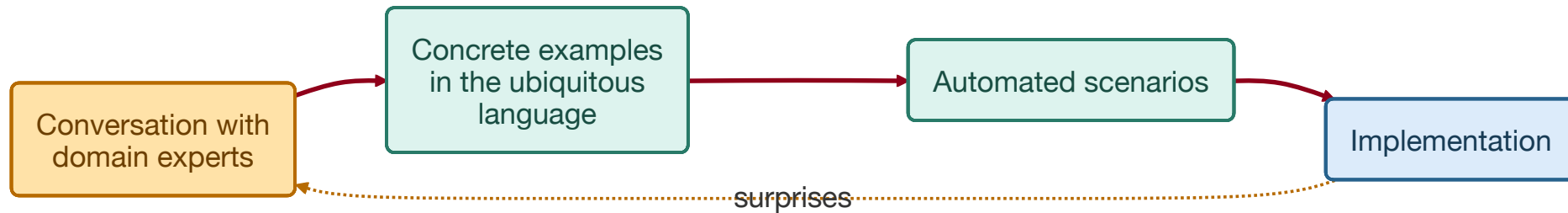


The domain expert cannot read `testOrderFlow3`. We want something else.

behavior-driven development

behavior-driven development (BDD)

behavior-driven development is a practice where the team agrees on concrete **examples** of the system's behavior before building it, writes the examples in a shared language, and automates them as tests.

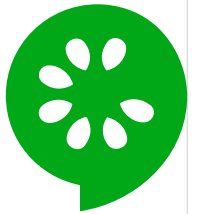


Examples are the shortest path to shared understanding

Gherkin: Given, When, Then

Gherkin is the language BDD tools use to write examples: cucumber.io/docs/gherkin

```
1 Feature: Place an order
2
3   Scenario: A customer orders items that are in stock
4     Given a customer with 2 "Wireless mouse" in the cart
5     And 10 "Wireless mouse" available in inventory
6     When the customer places the order with a valid credit card
7     Then the order is placed
8     And 8 "Wireless mouse" remain available
```



Given: the context, the state before

When: the action, one event

Then: the observable outcome

A use case maps directly onto Gherkin

Use case

- 1 Preconditions: signed in, cart not empty
- 2 1. The customer asks to check out.
- 3 2. The system shows the order summary.
- 4 3. The customer confirms and pays.
- 5 4. The system reserves the items.
- 6 5. The system places the order.

Scenario

- 1 **Given** a customer with 2 "Wireless mouse" in the cart
- 2 **And** 10 "Wireless mouse" available in inventory
- 3 **When** the customer places the order
- 4 **Then** the order is placed
- 5 **And** 8 "Wireless mouse" remain available

Use case

Gherkin

Preconditions, and the steps before the actor acts

Given

The actor's step that triggers the system

When

The system's response and the postcondition

Then

Each extension

Its own scenario

A Gherkin scenario enhances use cases with concrete values

A use case leads to many scenarios

The main success scenario and each extension become one scenario each.
The extension's condition goes into **Given**, its outcome into **Then**:

```
1 Feature: Place an order
2
3 Scenario: A customer orders items that are in stock
4   Given a customer with 2 "Wireless mouse" in the cart
5   And 10 "Wireless mouse" available in inventory
6   When the customer places the order with a valid credit card
7   Then the order is placed
8   And 8 "Wireless mouse" remain available
9
10 Scenario: An item in the cart is no longer in stock
11   Given a customer with 1 "Wireless mouse" in the cart
12   And 0 "Wireless mouse" available in inventory
13   When the customer places the order
14   Then the "Wireless mouse" is removed from the order summary
15
16 Scenario: The payment is declined
17   Given a customer with 2 "Wireless mouse" in the cart
18   And 10 "Wireless mouse" available in inventory
19   When the customer places the order with a declined credit card
20   Then the order is not placed
21   And 10 "Wireless mouse" remain available
```

Scenario data embed domain rules

The domain expert gives us more cases for the stock rule:

```
1  Scenario: Plenty of stock
2    Given a customer with 2 "Wireless mouse" in the cart
3    And 10 "Wireless mouse" available in inventory
4    When the customer places the order
5    Then the order is placed
6    And 8 "Wireless mouse" remain available
7
8  Scenario: Exactly enough stock
9    Given a customer with 5 "Wireless mouse" in the cart
10   And 5 "Wireless mouse" available in inventory
11   When the customer places the order
12   Then the order is placed
13   And 0 "Wireless mouse" remain available
14
15 Scenario: Not enough stock
16   Given a customer with 6 "Wireless mouse" in the cart
17   And 5 "Wireless mouse" available in inventory
18   When the customer places the order
19   Then the order is not placed
20   And 5 "Wireless mouse" remain available
```

The **behavior** is identical in all three. Only the **data** changed. The rule is buried under copies of the same five lines.

Scenario outline: separate behavior with data

```

1  Scenario Outline: Ordering depends on the available stock
2    Given a customer with <in cart> <item> in the cart
3    And <available> <item> available in inventory
4    When the customer places the order
5    Then the order is <outcome>
6    And <remaining> <item> remain available

```

```

8  Examples:
9    | in cart | available | outcome   | remaining | item           |
10   | 2        | 10        | placed    | 8          | Wireless mouse |
11   | 5        | 5         | placed    | 0          | Wireless mouse |
12   | 6        | 5         | not placed| 5          | Wireless mouse |

```

- The outline states the behavior **once**
- The **Examples** table holds the data
- Each row runs as a separate test
- The expert can just add **rows**
- Variables names become the contract data

The scenario uses the ubiquitous language

```
1 Scenario Outline: Ordering depends on the available stock
2   Given a customer with <in cart> <item> in the cart
3   And <available> <item> available in inventory
4   When the customer places the order
5   Then the order is <outcome>
6   And <remaining> <item> remain available
```

Every noun is a model concept:

**Customer, Cart, Product, Inventory,
Order, Payment method.**

Every verb describes use case step:

places the order, remain available.

A scenario that needs a word the domain expert does not use is a sign the model or the scenario is wrong.

Writing good scenarios

Avoid

```
1 Scenario: Checkout
2   Given I open the website
3   And I click "Cart"
4   And I click "Checkout"
5   And I fill in "4111 1111 1111 1111"
6   And I click "Pay"
7   Then I see "Thank you"
```

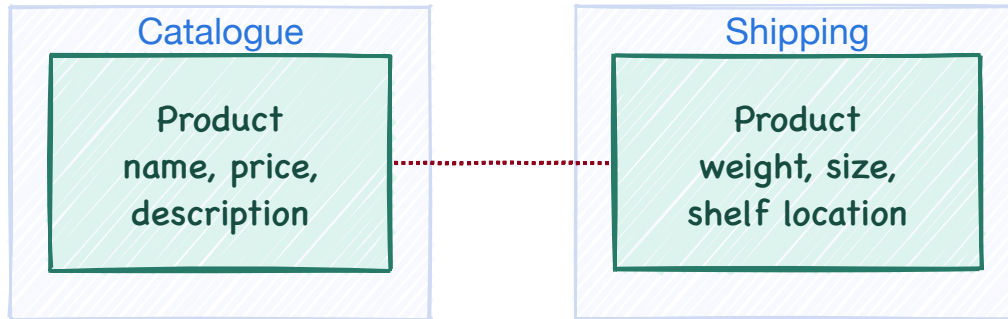
Describes the UI, not the behavior.
Breaks whenever a button moves.

Prefer

```
1 Scenario: Order for in-stock items
2   Given a customer with 2 "Wireless mouse" in the cart
3   And 10 "Wireless mouse" available in inventory
4   When the customer places the order
5   Then the order is placed
6   And 8 "Wireless mouse" remain available
```

- Declarative: **what**, not how
- Concrete data: 2, 10, 8
- One behavior, in domain terms
- Runs as a test (Cucumber binds each step to code, you will learn more about this in the labs)

Next: Decomposition and boundaries



One ubiquitous language for the whole shop turns out to be impossible.

“Product” means one thing to marketing and another to the warehouse.

These boundaries can be identified with **bounded context**.

Summary

- The **domain** is the problem space and the most stable part of the system. Code depends on it, never the reverse.
- A **domain model** keeps the concepts and rules that matter.
- **Use cases** capture behavior, and their extensions are where the rules hide.
- The **ubiquitous language** is one vocabulary for experts, documents, and code.
- **DDD** works at two levels: strategic (boundaries) and tactical (code inside a boundary).
- **Gherkin scenarios** follow the use case step by step and form an executable **contract**. **Scenario outlines** keep the behavior separated from the data.